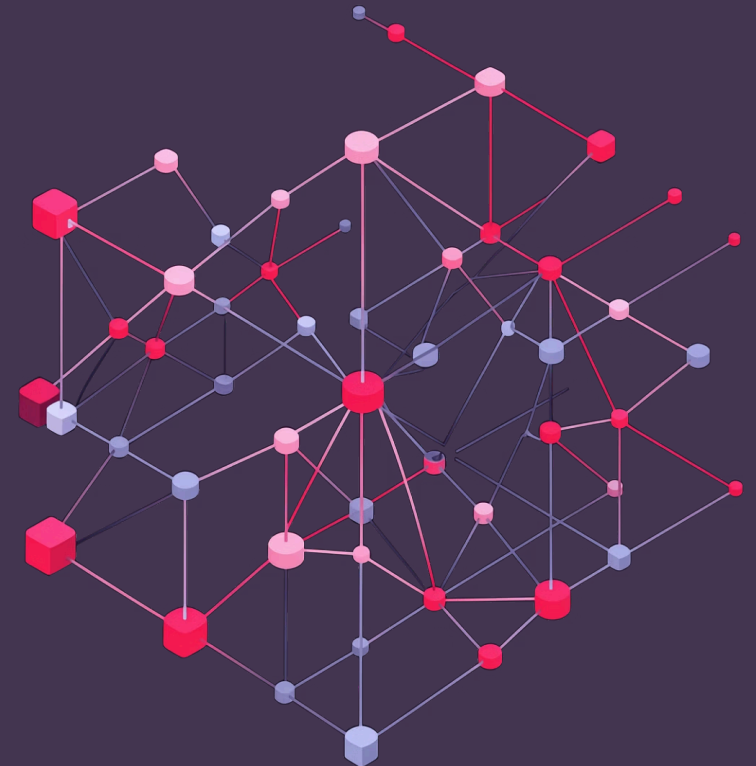


Breadth-First Search: Finding the Shortest Path

A visual guide to one of computer science's most elegant algorithms — and how it guarantees the fastest route through any network.



What is a Graph?

Nodes

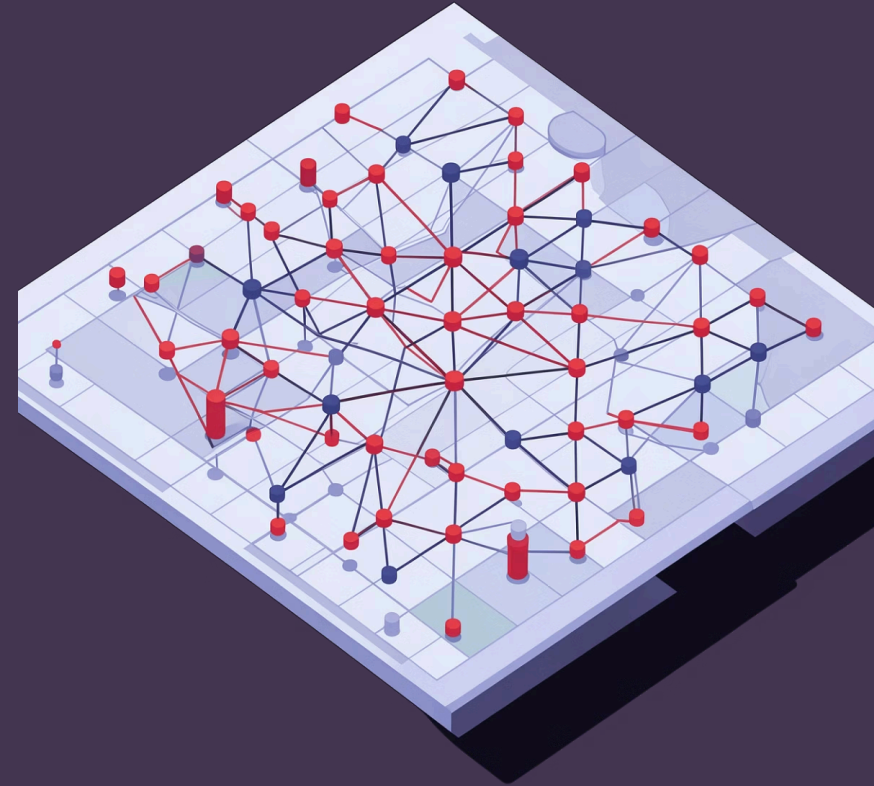
The individual points in a network — think of them as people, cities, or web pages.

Edges

The lines that connect nodes — representing relationships, roads, or links.

Real-World Examples

Social networks, city maps, websites, and friendship chains are all graphs in disguise.





The Shortest Path Problem

Imagine you're standing at **point A** and need to reach **point B**. Several routes wind through the network — but which one gets you there in the fewest steps?

The Challenge

Multiple routes exist — choosing wrong wastes time and resources.

The Guarantee

BFS always finds the shortest path — not just a path, but the optimal one.

How BFS Works: The Idea

BFS explores a graph the way ripples spread across water — moving outward one ring at a time.

1

Start

Begin at your starting node

2

Layer 1

Explore all neighbors **one step** away

3

Layer 2

Explore all neighbors **two steps** away

4

Destination

Stop the moment you reach your target

BFS Pseudocode

The algorithm in plain, readable steps — no programming experience required:

1 Initialize

Create a queue and add the starting node. Mark it as visited.

2 Process the Queue

While the queue is not empty, remove the first node.

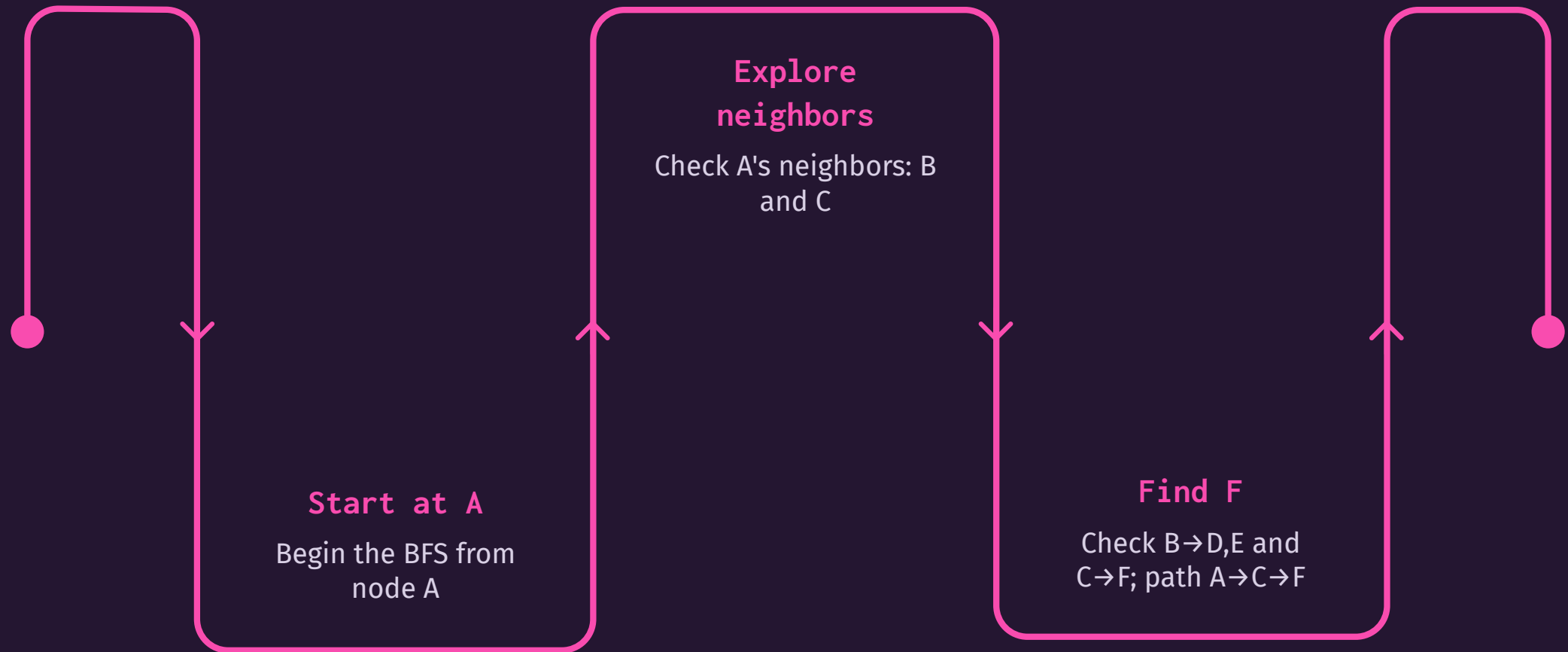
3 Check or Expand

If it's the destination, return the path. Otherwise, add all unvisited neighbors to the queue.

```
function BFS(start, goal):  
    queue = [start]  
    visited = {start}  
    parent = {start: null}  
  
    while queue is not empty:  
        node = queue.pop_front()  
  
        if node == goal:  
            return reconstruct(parent, goal)  
  
        for neighbor in node.neighbors:  
            if neighbor not in visited:  
                visited.add(neighbor)  
                parent[neighbor] = node  
                queue.push_back(neighbor)  
  
    return "No path found"
```

Visual Example: Finding a Friend

You're at node **A**, looking for node **F**. Watch BFS explore layer by layer:



📌 **Key insight:** BFS found F in just **2 steps** — and because it explores layer by layer, this is guaranteed to be the shortest possible path.



Why BFS Works



Layer-by-Layer Exploration

By expanding outward one level at a time, BFS never skips over a shorter route — the **first time it reaches a node is always the shortest path.**



The Queue Keeps Order

A **first-in, first-out** queue ensures closer nodes are always processed before farther ones — maintaining the layer guarantee.



No Revisiting Nodes

Marking nodes as visited prevents BFS from going in circles or processing the same node twice, keeping it efficient.

Real-World Applications

BFS powers some of the most widely-used technologies in the world:



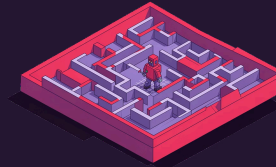
GPS Navigation

Route planning apps use BFS-style algorithms to find the fewest turns or shortest drive between two locations.



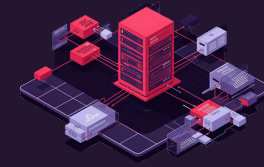
Social Networks

Platforms like LinkedIn calculate **degrees of separation** — how many connections stand between you and a stranger.



Game AI Pathfinding

Video game characters use BFS to navigate maps and find the player through complex, obstacle-filled environments.



Network Routing

Internet routers use BFS-based protocols to find the most efficient path for data packets across global networks.